

Python Lists Complete Guide

Core Concepts, Mutability, Methods, and Interview Questions

1. Understanding Core List Concepts

Mutability vs Immutability

The biggest difference between Strings and Lists is **Mutability**.

- **Strings are Immutable:** You cannot change them once created.
- **Lists are Mutable:** You can add, remove, and change items directly in the list without creating a brand-new list in memory.

The Variable Reference Trap

Because lists are mutable, you have to be careful with variables! If you do `b = a`, both variables point to the **exact same list** in memory. Changing `b` will change `a`. This is why the `copy()` method is so important.

Other Key Properties

- **Ordered:** Lists keep the exact order of the items you put into them. First item is at index `0`.
- **Dynamic:** They grow and shrink automatically as you add or remove items.
- **Allows Duplicates:** A list can contain the same value multiple times, e.g., `[1, 1, 2]`.

2. List Methods Code Examples

Below are practical, easy-to-understand code examples for every single list method.

1. `append(x)`

Adds one single item to the very end of the list.

```
cart = ["apple", "banana"]
cart.append("orange")
print(cart)
# Output: ['apple', 'banana', 'orange']
```

2. `extend(iterable)`

Takes another list (or iterable) and adds each of its items to the end of your current list.

```
team_a = ["Ali", "Sara"]
team_b = ["Talha", "Rehan"]

# Adds all members of team_b to team_a
team_a.extend(team_b)
print(team_a)
# Output: ['Ali', 'Sara', 'Talha', 'Rehan']
```

3. `insert(i, x)`

Adds an item at a specific index, pushing everything else to the right.

```
ranking = ["Gold", "Bronze"]

# Insert "Silver" at index 1 (the second spot)
ranking.insert(1, "Silver")
print(ranking)
# Output: ['Gold', 'Silver', 'Bronze']
```

4. remove(x)

Searches the list for a specific value and removes the **first** matching item it finds.

```
colors = ["red", "blue", "red", "green"]

# Removes only the first "red"
colors.remove("red")
print(colors)
# Output: ['blue', 'red', 'green']
```

5. pop(i)

Removes the item at a specific index and **returns** it so you can use it. If you don't provide an index, it pops the very last item.

```
tasks = ["Task 1", "Task 2", "Task 3"]

# Removes and grabs the last task
completed_task = tasks.pop()
print("Finished:", completed_task) # Output: Finished: Task 3
print("Remaining:", tasks)         # Output: Remaining: ['Task 1', 'Task 2']
```

6. clear()

Wipes out the entire list, leaving it completely empty.

```
shopping_list = ["milk", "bread", "eggs"]
shopping_list.clear()
print(shopping_list)
# Output: []
```

7. index(x)

Searches for a value and tells you the index number where it is located.

```
vowels = ["a", "e", "i", "o", "u"]
position = vowels.index("i")
print("The letter 'i' is at index:", position)
# Output: The letter 'i' is at index: 2
```

8. count(x)

Counts exactly how many times a specific value appears in your list.

```
dice_rolls = [1, 6, 3, 6, 6, 2]
sixes_rolled = dice_rolls.count(6)
print("You rolled a six", sixes_rolled, "times.")
# Output: You rolled a six 3 times.
```

9. sort()

Organizes the list in ascending order (alphabetical for text, lowest to highest for numbers). It changes the original list directly.

```
scores = [88, 45, 99, 72]
scores.sort()
print(scores)
# Output: [45, 72, 88, 99]
```

10. reverse()

Flips the entire list upside down, making the last item the first item.

```
countdown = [1, 2, 3, 4, 5]
countdown.reverse()
print(countdown)
# Output: [5, 4, 3, 2, 1]
```

11. copy()

Creates a shallow copy of the list. This prevents the memory trap where two variables control the same list!

```
original = ["pizza", "burger"]
my_copy = original.copy()

my_copy.append("fries")

# The original is safe from changes made to the copy!
print("Original:", original) # Output: ['pizza', 'burger']
print("Copy:", my_copy)      # Output: ['pizza', 'burger', 'fries']
```

3. Interview Questions Expanded

1. Difference between append and extend?

If you append ([1, 2]) to a list, it adds the whole list as a *single item* (creating a list inside a list). If you extend ([1, 2]), it extracts the items and merges them cleanly into the main list.

2. Difference between `sort()` and `sorted()`?

`sort()` modifies the existing list in place and returns `None`. `sorted(my_list)` leaves your original list completely alone and hands you back a brand new, sorted list.

3. Why does Python list not have `push()`?

Python developers designed lists to be flexible. Instead of adding a `push()` method specifically for "stack" behavior, they decided `append()` already does that exact job perfectly! (Last-in, first-out behavior is achieved using `append()` and `pop()`).